

# Git, Reviews and AI-Assisted Coding

The seven chapters our team uses every single day, in plain language. Keep this one open while you build.

- 4 Git and version control
- 5 Writing good commits
- 6 Branching
- 7 Pull requests and code review
- 8 Merge conflicts
- 9 .gitignore, config and secrets
- 10 Working with AI

Every day you will branch, commit, push, open a pull request, review someone else's work, and fix a conflict. This booklet is only about that.

**Manish**

Team lead

**Anuradha**

Window 2

**Aaradhya**

Window 3

**Divyanshi**

Window 4

Taken from Parts 4–10 of the full **Team Software Engineering Practice Playbook**. Same technical meaning, shorter and simpler. Part numbers match the big book, so you can always look up more detail there.

# What is in this booklet

---

|                                            |           |
|--------------------------------------------|-----------|
| <b>Git and version control</b>             | <b>3</b>  |
| 4.1 The four locations                     | 3         |
| 4.2 Words you need to know                 | 4         |
| 4.3 A commit is a snapshot with a parent   | 4         |
| 4.4 A branch is a pointer, not a copy      | 5         |
| 4.5 The commands you actually need         | 6         |
| 4.6 Things that go wrong, and what to do   | 7         |
| 4.7 Undoing things safely                  | 7         |
| 4.8 Your daily loop                        | 8         |
| <b>Writing good commits</b>                | <b>9</b>  |
| 5.1 What a good commit looks like          | 9         |
| 5.2 Bad and good, side by side             | 10        |
| 5.3 The message format we use              | 10        |
| 5.4 The two-commit rule (worth real marks) | 12        |
| 5.5 Our 12 commit rules                    | 12        |
| <b>Branching</b>                           | <b>14</b> |
| 6.1 Our model in one picture               | 14        |
| 6.2 Naming branches                        | 14        |
| 6.3 The life of a branch                   | 15        |
| 6.4 Why we are not using GitFlow           | 16        |
| 6.5 When two people need the same file     | 16        |
| <b>Pull requests and code review</b>       | <b>18</b> |
| 7.1 Before you open it                     | 18        |
| 7.2 Writing the description                | 18        |
| 7.3 The four things a reviewer checks      | 19        |
| 7.4 The review checklist                   | 20        |
| 7.5 How to write a review comment          | 20        |
| 7.6 Approve, request changes, or wait      | 21        |
| 7.7 Handling review comments as the author | 21        |
| <b>Merge conflicts</b>                     | <b>23</b> |
| 8.1 Why conflicts happen                   | 23        |
| 8.2 Reading the markers                    | 23        |
| 8.3 A real conflict, resolved              | 25        |
| 8.4 Our 9-step procedure                   | 26        |
| 8.5 How to have fewer conflicts            | 27        |

---

|                                             |           |
|---------------------------------------------|-----------|
| <b>.gitignore, config and secrets</b>       | <b>28</b> |
| 9.1 Config is an input, not a constant      | 28        |
| 9.2 What .gitignore is — and what it is not | 29        |
| 9.3 When a secret is already committed      | 31        |
| 9.4 NEVER COMMIT THESE                      | 31        |
| 9.5 The same rule applies to AI prompts     | 32        |
| 9.6 Three buckets, three different fates    | 33        |

---

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>Working with AI</b>                          | <b>34</b> |
| 10.1 Two ways to work with an assistant         | 34        |
| 10.2 Why it invents functions that do not exist | 34        |
| 10.3 Context engineering                        | 35        |
| 10.4 The agent context file                     | 36        |
| 10.5 Plan → Spec → Generate → Validate          | 37        |
| 10.6 Context rot                                | 39        |
| 10.7 What to use it for — and what to check     | 39        |
| 10.8 AI code review checklist                   | 40        |
| 10.9 Learning, not depending                    | 41        |

## 04

## Git and version control

Learn the model, not the commands. Then the commands explain themselves.

**In this part:** the four places your code lives · what a commit really is · why a branch is cheap · every command you actually need · how to undo things safely

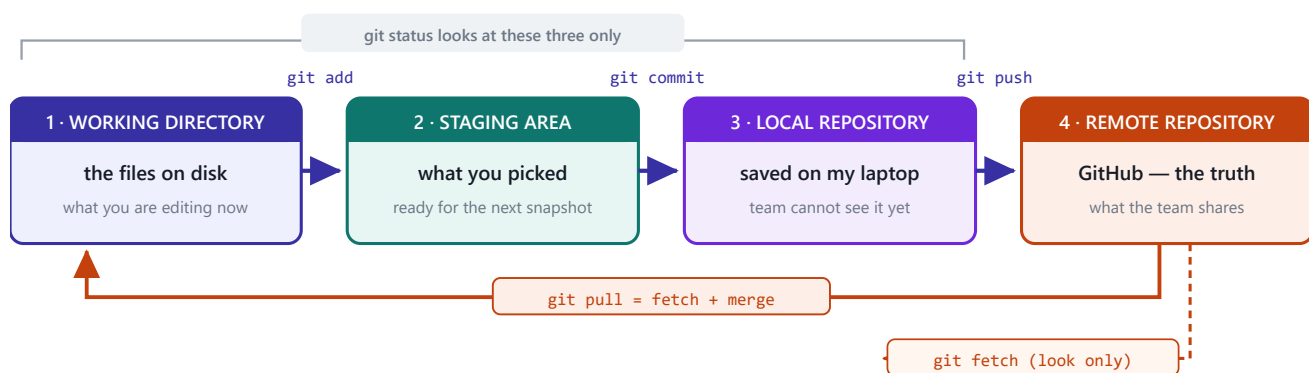
**“Almost every confusing git error is a statement that two of these four locations disagree.”**

Lecture T0 · L1

That one sentence is worth more than fifty memorised commands. Learn the four locations, and an error stops being scary. It becomes a question with a short list of answers.

## 4.1 The four locations

*A clean `git status` says nothing about GitHub. That is a different question.*



Four places. Three commands move code forward. Two bring it back.

**THE CLASSIC MISTAKE**

**What you see:** `git status` says *nothing to commit, working tree clean*, but your teammate cannot see your change on GitHub.

**Which two disagree?** Your **local repository** and the **remote**.

**Why:** `git status` only compares locations 1, 2 and 3. It never checks GitHub.

**Fix:** `git push`.

## 4.2 Words you need to know

| Word                     | What it actually is                                                                         |
|--------------------------|---------------------------------------------------------------------------------------------|
| Working tree             | The files on disk right now. Git is just watching them.                                     |
| Untracked                | Git has never seen this file.                                                               |
| Modified                 | Git has seen it before. The file on disk is now different from the last snapshot.           |
| Staged                   | A copy of the file is frozen and waiting for the next commit.                               |
| Commit                   | A snapshot plus a pointer to the commit before it.                                          |
| Local repository         | The hidden <code>.git</code> folder. All your saved snapshots.                              |
| Remote repository        | Another repository on a server. GitHub runs the <i>same format</i> you have locally.        |
| <code>origin</code>      | Just a <i>name</i> for a remote. Not a place. It is whichever remote you set up first.      |
| HEAD                     | A pointer saying "where I am right now". Normally it points at a branch.                    |
| Branch                   | A small file holding one commit ID. Nothing more.                                           |
| <code>origin/main</code> | Your <i>memory</i> of where GitHub's <code>main</code> was, the last time you talked to it. |

**EASY WAY TO REMEMBER IT**

The working directory is **backstage** — messy, everything you are touching. The staging area is **the lit stage** — only what you chose to show. `git commit` takes the photo. Anything still backstage is not in the photo.

## 4.3 A commit is a snapshot with a parent

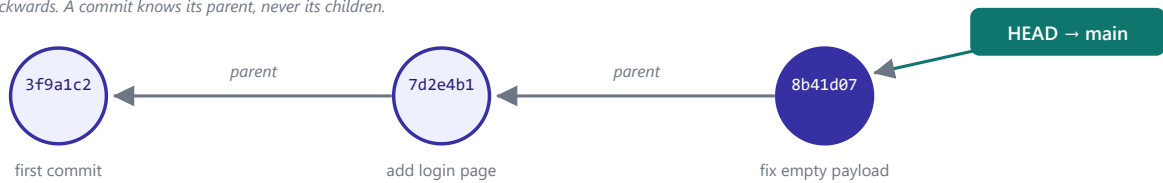
Three facts. Everything else follows from them.

- 1 Every commit points to its parent.**  
The commit that came before it. A merge commit points to two.
- 2 Follow the parents back and you get the whole history.**  
In order. That is all history is — a chain of pointers.

### 3 Commits cannot be edited, only replaced.

Change anything in a commit and its ID changes. So “editing history” always means making new commits and throwing the old ones away.

Arrows point backwards. A commit knows its parent, never its children.



The branch label just sits on the newest commit.

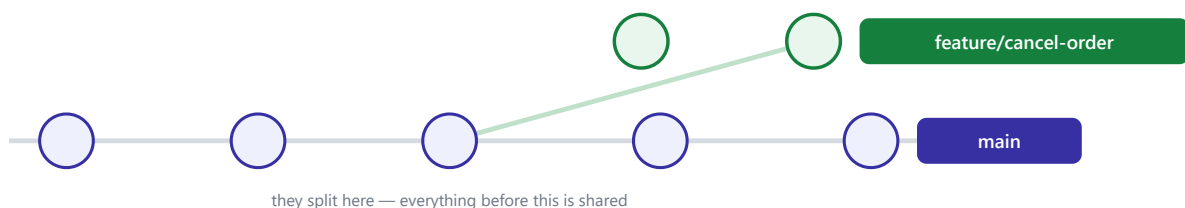
## 4.4 A branch is a pointer, not a copy

A branch is a small file holding one commit ID. When you commit, git rewrites that one file.

**No files are copied.**

Old tools like SVN copied the whole folder for every branch. It was so slow that teams avoided branching and worked on top of each other instead. Git made branching free, so teams stopped avoiding it. **Our whole pull-request workflow only exists because branching is cheap.**

Two labels, one shared history. Making the label took no time because it is one tiny file.



Branching costs nothing, so branch for everything.

## 4.5 The commands you actually need

This is the full list for our project. Read the “what changes” column — that is the part people get wrong.

| Command                                 | What it does                                               | What changes                                                                                 |
|-----------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>git status</code>                 | Asks git what changed.                                     | Nothing. Safe to run 100 times a day, and you should.                                        |
| <code>git diff</code>                   | Shows edits you have <i>not</i> staged yet.                | Nothing.                                                                                     |
| <code>git add &lt;file&gt;</code>       | Freezes that file’s current content into the staging area. | File moves to “Changes to be committed”. Other files are untouched.                          |
| <code>git diff --cached</code>          | Shows exactly what you are about to commit.                | Nothing. <b>Run this before every commit.</b>                                                |
| <code>git commit -m "..."</code>        | Saves the staged content as a snapshot.                    | New commit exists on your laptop. The branch pointer moves. <b>Nothing is sent anywhere.</b> |
| <code>git log</code>                    | Walks back through history.                                | Nothing. Press <code>q</code> to quit.                                                       |
| <code>git branch</code>                 | Lists branches.                                            | Nothing.                                                                                     |
| <code>git switch -c &lt;name&gt;</code> | Creates a branch and moves to it.                          | New pointer at the same commit. No files copied.                                             |
| <code>git switch &lt;name&gt;</code>    | Moves to an existing branch.                               | Your files on disk change to match that branch.                                              |
| <code>git fetch</code>                  | Downloads new commits from GitHub.                         | <code>origin/main</code> updates. <b>Your files do not change.</b>                           |
| <code>git pull</code>                   | Fetch, then merge into your branch.                        | <b>Your files do change.</b>                                                                 |
| <code>git push</code>                   | Sends your commits to GitHub.                              | The remote branch moves. Teammates can now see it.                                           |
| <code>git merge &lt;branch&gt;</code>   | Joins two histories.                                       | A merge commit with <b>two</b> parents — or a conflict (Part 8).                             |
| <code>git stash</code>                  | Parks unfinished work in a drawer.                         | Working tree goes clean. <code>git stash pop</code> brings it back.                          |

### fetch vs pull — the one-letter difference

#### git fetch — looks

- Downloads new commits
- Updates `origin/main`
- **Your files stay exactly as they are**

*Safe even when you are half way through an edit.*

#### git pull — looks and merges

- Does a fetch...
- ...then merges into your branch
- **Your files change**

*Do this when you are ready to take other people’s work.*

## 4.6 Things that go wrong, and what to do

### STAGED AND MODIFIED AT THE SAME TIME

**What you see:** one file appears under *both* headings in `git status`.

**Why:** you staged it, then kept editing. Staging took a *copy*. Your newer edits are not in that copy.

**Fix:** run `git add` on that file again.

### PUSH REJECTED — DO NOT FORCE

**What you see:** “failed to push some refs”.

**Why:** GitHub has commits that you do not have.

**Fix:** `git pull` first, then push again.

**Never** reach for `git push --force`. It can delete a teammate’s work.

### “I LOST MY COMMITS” — DETACHED HEAD

**How it happens:** you ran `git checkout <some-old-id>` to look at an old version, then started editing and committing there.

**Why it breaks:** no branch is pointing at those new commits. Switching away seems to delete them.

**Get them back:** `git reflog` lists where HEAD has been. Find the ID, then `git switch -c rescue <id>`.

**Avoid it:** never commit while detached. Look, then switch back.

## 4.7 Undoing things safely

Ordered from safest to most dangerous. Read the risk column before you type anything.

| You want to...                             | Command                                        | Risk                                                                                    |
|--------------------------------------------|------------------------------------------------|-----------------------------------------------------------------------------------------|
| Unstage a file, keep the edit              | <code>git restore --staged &lt;file&gt;</code> | None.                                                                                   |
| Throw away an uncommitted edit             | <code>git restore &lt;file&gt;</code>          | <b>The edit is gone.</b> Git never saw it, so it cannot bring it back.                  |
| Fix the last message (not pushed)          | <code>git commit --amend</code>                | Makes a new commit with a new ID. Fine only if nobody pulled it.                        |
| Undo a commit already on <code>main</code> | <code>git revert &lt;id&gt;</code>             | <b>Safest option.</b> Adds a new commit that reverses the change. History stays intact. |
| Move your branch back, keep changes        | <code>git reset --soft &lt;id&gt;</code>       | Recoverable.                                                                            |
| Move back and delete everything            | <code>git reset --hard &lt;id&gt;</code>       | <b>Destroys uncommitted work permanently.</b> Never on a shared branch.                 |



**TEAM RULE**

If it has been pushed, we use `git revert`. Never `reset` plus force-push. A revert is a normal commit — it gets reviewed, it shows in history, and it cannot destroy anyone's work.

If you think you need a force-push, **stop and ask the team first**.

## 4.8 Your daily loop

Copy this. It is the same eight steps every single day.

```
# 1. start from a fresh main
$ git switch main
$ git pull

# 2. one branch for one piece of work
$ git switch -c feature/cancel-order

# 3. ...edit your files...

# 4. look before you stage
$ git status
$ git diff

# 5. stage on purpose, then check what you staged
$ git add app/orders.py tests/test_orders.py
$ git diff --cached

# 6. commit with a real message
$ git commit -m "feat(orders): allow cancelling an unshipped order"

# 7. publish the branch
$ git push -u origin feature/cancel-order

# 8. open the pull request, then stay around for review comments
```

**LIKELY VIVA QUESTION**

**"Why does git need a staging area?"** So you can commit only *some* of what you changed. That is what makes one-idea commits possible.

**"Why is making a branch instant?"** It writes one small file holding a commit ID. Nothing is copied.

**"fetch or pull?"** Pull is fetch plus a merge that changes your files.

## 05

## Writing good commits

You write a commit message once. People read it for years.

**In this part:** what makes a commit good · bad vs good examples · the message format we use · the two-commit rule that is worth marks · our 12 commit rules

## Is git a save button, or a conversation? — A conversation. Read by people.

Lecture T0 · L0

Your teammates read your commit history far more often than you do. So does a reviewer. So does an examiner. It is the first thing anyone sees.

### 5.1 What a good commit looks like

A commit message has two jobs. The **subject says what**. The **body says why**. The diff already shows the details, so do not repeat them in words.

```
fix(auth): reject expired refresh tokens
```

Refresh tokens issued before the key rotation were still being accepted after revocation. Reject anything signed with the retired key instead of trusting the expiry claim alone.

| Rule                           | What it means when you are typing                                                               |
|--------------------------------|-------------------------------------------------------------------------------------------------|
| One idea per commit            | The whole commit does one thing. <i>If the subject needs the word "and", it is two commits.</i> |
| Small enough to review         | A teammate can read every changed line in a few minutes.                                        |
| A subject that means something | Someone who was not there can tell what happened without opening the diff.                      |
| Imperative wording             | "reject", not "rejects" or "rejected". Read it as an order the commit carries out.              |
| A body when it helps           | Explain why the bug happened, or why you chose this approach. Small commits need no body.       |

**THE ONE-SECOND TEST**

Put your subject in this sentence: *"If applied, this commit will \_\_\_\_"*

✓ ...will **reject expired refresh tokens**. Reads fine.

✗ ...will **fixed the login bug**. Does not read.

## 5.2 Bad and good, side by side

### ✗ Bad

updates  
changes  
final fix  
stuff  
fixed it  
work in progress

None of these tell a reviewer anything. In six weeks they will not tell *you* anything either.

### ✓ Good

feat(auth): add refresh token rotation  
fix(cart): reject quantities below one  
test(auth): cover expired refresh tokens  
docs(api): document auth endpoints  
refactor(orders): extract price calc  
chore(deps): pin express to 4.21.0

Each one names an area, states an action, and stands alone.

### The same change, written three ways

| Quality | Message                                                                                                                   | What the reviewer thinks                                                    |
|---------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Bad     | fix                                                                                                                       | "Fix what? Now I have to read all 240 changed lines."                       |
| Better  | fix login bug                                                                                                             | "Which login bug? Three were open."                                         |
| Good    | fix(auth): reject tokens signed with the retired key — plus a body saying the old expiry check let revoked tokens through | "I know exactly what to check, and why the easy fix would not have worked." |

## 5.3 The message format we use

```
<type>(<scope>): <what it does, lower case, no full stop>
<blank line>
<body: why, not what>
```

| Type     | Use it for                      | Example                                     |
|----------|---------------------------------|---------------------------------------------|
| feat     | New behaviour a user can see    | feat(cart): allow saving a cart for later   |
| fix      | A bug repair                    | fix(cart): reject quantities below one      |
| test     | Tests only, behaviour unchanged | test(auth): cover expired refresh tokens    |
| docs     | Documentation only              | docs(api): document auth endpoints          |
| refactor | Cleaner code, same behaviour    | refactor(orders): extract price calculation |
| chore    | Tooling, dependencies, config   | chore(deps): pin express to 4.21.0          |

| Type | Use it for              | Example                                 |
|------|-------------------------|-----------------------------------------|
| perf | A change made for speed | perf(feed): add index on posts(user_id) |

**Scope** is the area you touched: `auth`, `cart`, `orders`, `db`, `ci`. Agree the list in week 1 and put it in `AGENTS.md`, so all four of us use the same words.

**BE HONEST ABOUT WHERE THIS COMES FROM**

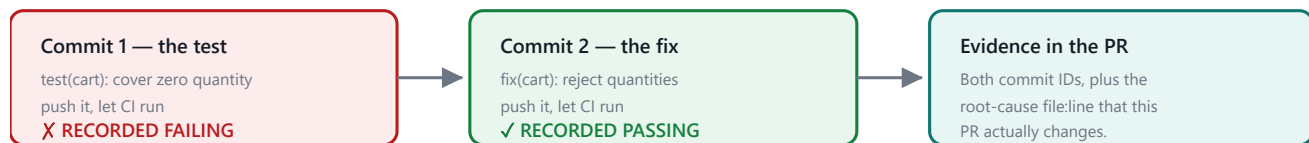
Every lecture example uses this `type(scope): subject shape`. But no delivered lecture names the “Conventional Commits” standard or says you must use it.

So: the **format shown in lectures** is course material. **Formally adopting it** is our team’s choice. If Sir specifies something different later, his version wins.

## 5.4 The two-commit rule (worth real marks)

This is 30% of every milestone, so get it exactly right. Every graded bug fix is **two commits**: a test recorded as *failing*, then the fix recorded as *passing*.

The order is checked from the CI record, not from your files.



Writing the fix first and the test after gives the same files — and fails the check. The record is the proof, not the code.

```

# 1. turn the bug into a test, and let it FAIL
$ git add tests/test_cart.py
$ git commit -m "test(cart): cover rejection of zero quantity"
$ git push          # CI records this commit RED

# 2. now fix it, and let the same test PASS
$ git add src/cart.py
$ git commit -m "fix(cart): reject quantities below one"
$ git push          # CI records this commit GREEN
  
```

### DO NOT SQUASH THESE TWO

Squashing them into one commit destroys the evidence the milestone is graded on. For a graded bug fix, **keep both commits** and say so in the PR description.

### WHY THE FAILING RUN MATTERS

A test written *after* the fix has never been seen to fail. It might test nothing useful and you would never know. Watching it go red first is the only proof it really covers the bug.

## 5.5 Our 12 commit rules

Numbered so we can point at them in review ("C3 — please rename this").

| #  | Rule                                                          | Why                                                                          |
|----|---------------------------------------------------------------|------------------------------------------------------------------------------|
| C1 | Do not mix unrelated changes                                  | Mixed commits cannot be reverted cleanly or reviewed honestly. <i>Course</i> |
| C2 | Never commit a secret                                         | Hardcoded into source means hardcoded into history, forever. <i>Course</i>   |
| C3 | No vague subjects: <i>updates, changes, fix, stuff, final</i> | The subject is the only thing most people read. <i>Course</i>                |
| C4 | If the subject needs "and", split it                          | Fastest reliable test for "one idea". <i>Course</i>                          |

| #   | Rule                                                    | Why                                                                                                             |
|-----|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| C5  | Use <code>type(scope): subject</code>                   | Matches every lecture example and makes history scannable. <i>Our team</i>                                      |
| C6  | Body explains <i>why</i> when the reason is not obvious | The diff already shows what. <i>Course</i>                                                                      |
| C7  | Keep commits small enough to review                     | A reviewer checks whether a change is small enough to reason about. <i>Course</i>                               |
| C8  | Do not commit <code>node_modules</code> or build output | They are huge and can be regenerated. Lockfiles <i>are</i> committed; installed packages are not. <i>Course</i> |
| C9  | Never knowingly push broken code to a shared branch     | One exception: the deliberate red commit in the two-commit rule, explained in the PR. <i>Our team</i>           |
| C10 | Run <code>git diff --cached</code> before every commit  | Only way to be sure what you are about to snapshot. <i>Our team</i>                                             |
| C11 | Commit under your own name and e-mail                   | The course grades named people. Attribution is not decoration. <i>Our team</i>                                  |
| C12 | Commit often locally, push when the branch makes sense  | Local commits are free and reversible. A pushed branch is a public statement. <i>Our team</i>                   |

**TEAM RULE**

Anyone may ask an author to rewrite a commit message on a branch that is **not yet merged**, and the author does it without arguing. It takes thirty seconds now and saves an hour of confusion in week nine.

**LIKELY VIVA QUESTION**

**"Show me your commit history."** A history of `fix`, `fix2`, `final` says you worked alone and nobody depended on you. A history of scoped, single-purpose commits says you have worked on a team. That signal costs nothing but attention.

## 06

## Branching

The simplest thing that works for four people. On purpose.

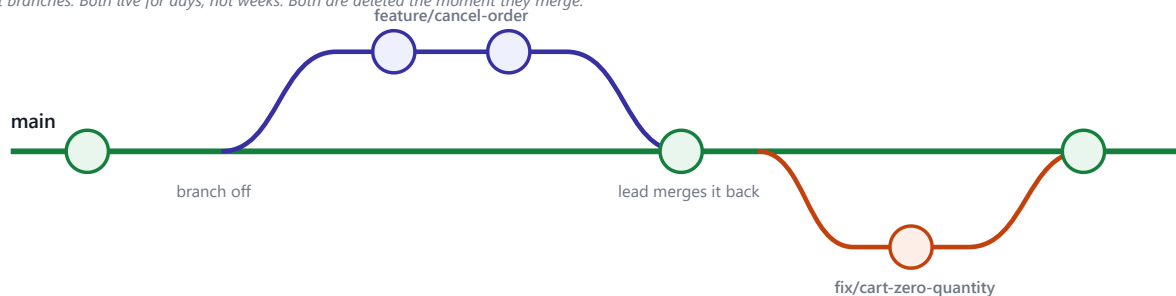
**In this part:** our branching model · naming branches · the full life of a branch · why we skip GitFlow  
· what to do when two people need the same file

## 6.1 Our model in one picture

One long-lived branch. Short branches off it. Everything comes back through a pull request.

|                          |                                         |
|--------------------------|-----------------------------------------|
| main                     | ← always works, always green, protected |
| ├ feature/cancel-order   | ← new behaviour                         |
| ├ fix/cart-zero-quantity | ← a bug repair                          |
| ├ refactor/price-module  | ← tidy-up, same behaviour               |
| ├ docs/architecture-doc  | ← documentation only                    |
| └ chore/pin-express      | ← tooling and dependencies              |

Two short branches. Both live for days, not weeks. Both are deleted the moment they merge.



## 6.2 Naming branches

| Prefix    | Use it for                       | Example                |
|-----------|----------------------------------|------------------------|
| feature/  | New behaviour a user can see     | feature/cancel-order   |
| fix/      | A bug repair                     | fix/cart-zero-quantity |
| refactor/ | Internal tidy-up, same behaviour | refactor/price-module  |
| docs/     | Documentation only               | docs/rotation-calendar |
| chore/    | Tooling, CI, dependencies        | chore/pin-express      |

**✓ Do**

- Lower case, hyphens between words
- Describe *the work*
- Short enough to type
- Add the issue number if we use one: `fix/42-cart-zero`

**✗ Don't**

- Name it after yourself: `divyanshi-branch`
- Use spaces or capitals
- Write a sentence: `fix/the-bug-anuradha-found-on-friday`

## 6.3 The life of a branch

### CREATE IT — ALWAYS OFF A FRESH MAIN

```
$ git switch main
$ git pull                      # this is the line people skip
$ git switch -c feature/cancel-order
```

**COMMON MISTAKE**

Branching off a `main` that is three days old. Everything works locally, and then the merge is painful because you are absorbing three days of other people's work all at once.

`git pull` first, every time. It takes two seconds.

### KEEP IT FRESH WHILE YOU WORK

If your branch lives more than about two days, bring `main` into it. You then meet conflicts early, in small pieces.

```
$ git switch main && git pull
$ git switch feature/cancel-order
$ git merge main                # fix conflicts NOW, not at PR time
```

### PUBLISH IT, THEN DELETE IT

```
$ git push -u origin feature/cancel-order

# after the lead merges it on GitHub:
$ git switch main && git pull
$ git branch -d feature/cancel-order      # delete local
$ git push origin --delete feature/cancel-order # delete remote
```

GitHub can delete the remote branch automatically when you merge. Turn that on once and forget it.

**TEAM RULE**

**One branch, one purpose, one owner, under a week of life.**

If a branch is still open after a week, it is too big. Split it and merge the part that is finished.



## 6.4 Why we are not using GitFlow

GitFlow adds `develop`, `release/*` and `hotfix/*` branches on top of what we have. It was built for teams shipping numbered versions on a schedule, supporting several versions at once.

**We are four people shipping continuously from one repository.**

| GitFlow adds                    | To solve                                               | Do we have that problem?                                       |
|---------------------------------|--------------------------------------------------------|----------------------------------------------------------------|
| A <code>develop</code> branch   | Keeping <code>main</code> at the last released version | No. Our <code>main</code> is what is deployed.                 |
| <code>release/*</code> branches | Stabilising a version while new work continues         | No. We do not run parallel versions.                           |
| <code>hotfix/*</code> branches  | Patching an old release                                | No. There is only one release: the current one.                |
| More places to merge            | —                                                      | It buys us more conflicts and more “which branch was that on?” |

**Complexity you do not need is not free. It costs attention you could have spent on the actual system.**

This is the same idea the course applies to architecture: *pick the simplest thing that meets the requirements, and be able to say why*. Over-engineering the git workflow for four people is the version-control version of reaching for microservices on day one.

### WHERE THIS RULE COMES FROM

No delivered lecture prescribes a branching model. The course outline only requires that all graded code reaches the default branch **through pull requests**, and that branch protection is on.

The trunk-based model above is **our team's choice**. It meets that requirement with the least machinery.

## 6.5 When two people need the same file

This is where four-person teams generate most of their pain. Three options, best first.

### 1 Take turns.

The lead decides who goes first. The second person waits, then branches off the new `main`. Boring, and almost always right.

### 2 Split the interface first.

One tiny PR that only adds the function signature or the module boundary, merged fast. Then both people build behind it without touching the same lines.

### 3 Both branches, merge main daily.

Only when the work genuinely cannot be sequenced. Expect conflicts, and fix them daily while they are small.

#### LIKELY VIVA QUESTION

**“What branching strategy does your team use, and why?”**

Weak answer: naming a famous model. Strong answer: name the model *and the reason* — “*trunk-based with short branches, because we are four people deploying continuously from one repository. GitFlow’s release branches would solve a problem we do not have.*”

## 07

# Pull requests and code review.

A pull request asks for feedback. It is not a merge button.

**In this part:** what to check before you open one · how to write the description · the four things reviewers check · the review checklist · how to write a comment · when not to merge

**Would you cross a bridge nobody inspected? Then do not merge code nobody reviewed.**

Lecture T0 · L0

A pull request is where your work becomes visible: the branch, the commits, the changed files, and most importantly **a written explanation from you**.

It is also **graded**. The T0 deliverable is a branch, commits with real messages, a written PR description, **one review comment given to a teammate**, and a merge.

## I 7.1 Before you open it

Do all of this *before* you ask a teammate to spend their time.

- ☐ I read my own diff, top to bottom, **on the PR page** — not in my editor
- ☐ The branch has only this one piece of work — no stray files, no debug prints, no commented-out code
- ☐ Tests exist for the new behaviour, and I ran them
- ☐ I started the app and used the feature like a user would
- ☐ No key, token, password or connection string appears in the diff
- ☐ The commit messages make sense to someone who was not in the room
- ☐ `main` is merged in, so the PR is not stale
- ☐ The description is written, and it says *why*, not just *what*
- ☐ If this is a graded bug fix, both commits are there and named

## I 7.2 Writing the description

Copy this shape every time.

```

## What
Users can cancel an order that has not shipped. A cancelled order is
refunded within 24 hours by the existing refund job.

## Why
Milestone 2 feature brief, point 3. Support currently cancels by hand.

## How it works
- POST /orders/:id/cancel checks status, then writes cancellation_requested_at
- The existing nightly refund job picks up rows with that column set
- Cancelling twice does nothing: the second call returns 200 and changes nothing

## How I tested it
- Unit: tests/orders/test_cancel.py (6 cases incl. already-shipped, double-cancel)
- By hand: docker compose up, cancelled an order, checked the row and the refund

## Risk
Touches the orders read path. If the status check is wrong, a shipped order
could be cancelled. Covered by test_cancel_rejects_shipped.

## Evidence record          (only for graded bug fixes)
Observable change : cart accepted quantity 0 and charged for nothing
Root cause       : src/cart.py:88
Failing commit   : 3f9a1c2
Passing commit   : 8b41d07
Tests covering it : tests/test_cart.py::test_rejects_zero_quantity

```

#### WHY THIS SHAPE WORKS

**What** and **Why** let the reviewer judge whether this is even the right change. **How I tested it** turns “it works” into evidence. **Risk** tells the reviewer where to look hardest — and being honest here makes reviews *faster*, not more critical.

## 7.3 The four things a reviewer checks

Learn these four. They are the backbone of every review you will ever give or receive.

### 1 • Correctness

Does it do what it says it does? Read the code against the description, not against what you assumed.

### 2 • Blast radius

What breaks if this is wrong? A change to login is not the same risk as a change to a help page.

### 3 • Test coverage

**Proof, not promises.** Is the new behaviour actually tested, including the edge cases?

### 4 • Cognitive complexity

Can the next person follow this? Is it small enough to hold in your head?

## 7.4 The review checklist

Work down this list. The four course items are marked; the rest are industry practice or our own additions.

| Check            | Source      | Ask yourself                                                                                                                                          |
|------------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Correctness      | Course      | Trace one happy path and one failure path by hand.                                                                                                    |
| Tests            | Course      | Would the test fail if the fix were removed? Are zero, one, empty, maximum and duplicate covered?                                                     |
| Security         | Later topic | Is input checked? Are queries parameterised? Is the caller allowed to touch <i>this specific object</i> , not just logged in? Any secret in the diff? |
| Blast radius     | Course      | What else calls this? What happens to existing data? If it breaks at 2 a.m., what does that look like?                                                |
| Complexity       | Course      | Could you explain this function in one sentence? Are the names honest?                                                                                |
| Documentation    | Our team    | Does <code>AGENTS.md</code> , <code>.env.example</code> or the README need an update?                                                                 |
| Migration safety | Later topic | Any schema change? Is it reversible? Does old code still work during the deploy?                                                                      |
| Observability    | Later topic | If this breaks in production, is there a log line that would tell us?                                                                                 |
| Configuration    | Course      | New setting read from the environment? Is it in <code>.env.example</code> with a safe default?                                                        |
| Scope            | Our team    | Is everything here part of one change? Unrelated improvements go in their own PR.                                                                     |

## 7.5 How to write a review comment

A useful comment says **what**, **why**, and **how sure you are**. Start every comment with one of four tags so the author knows what to do with it.

| Tag                      | Meaning                               | Example                                                                                                     |
|--------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>blocking:</code>   | Must change before merge              | <i>blocking: this query puts <code>user_id</code> straight into the SQL string. Please parameterise it.</i> |
| <code>question:</code>   | I do not understand yet               | <i>question: what happens if cancel is called twice in the same second?</i>                                 |
| <code>suggestion:</code> | Better, but I will approve either way | <i>suggestion: a helper here would remove the nesting. Your call.</i>                                       |
| <code>praise:</code>     | Genuinely good — say so               | <i>praise: the double-cancel test is exactly the case I was worried about.</i>                              |

**X Unhelpful**

- “This is wrong.”
- “I would have done it differently.”
- “LGTM” on a 600-line diff, two minutes after it opened
- Rewriting the author’s code in the comment box
- Style opinions a linter should handle

**✓ Helpful**

- Name the line, the risk, and the fix
- Ask when unsure instead of asserting
- Separate “must” from “nice”
- Point at a concrete failing input
- Say what is good, not only what is wrong

## 7.6 Approve, request changes, or wait

| Decision               | When                                                                                                                                                             |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Approve</b>         | You understand it, you believe it is correct, tests cover it, and anything left is a suggestion you are happy to leave to the author.                            |
| <b>Request changes</b> | Something is wrong, unsafe, untested, or you genuinely cannot follow it. <b>“I cannot follow this” is a valid reason</b> — complexity is one of the four checks. |
| <b>Comment only</b>    | You have questions but no verdict yet, or you only reviewed part of it. Say which part.                                                                          |

**DO NOT MERGE WHEN...**

- CI is red
- Nobody has reviewed it
- You are not the current window’s lead
- There are unanswered `blocking:` comments and the author is asleep
- It contains a secret — even “only a test key”
- It has a schema change nobody checked for reversibility
- **You do not understand what it does.** Merging something you cannot explain makes you accountable for it.

## 7.7 Handling review comments as the author

- 1 Read all of them before replying.**  
Half of them usually answer each other.
- 2 Reply to every one.**  
Even “done” or “good catch, fixed in 2a91f4c”. Silence looks like disagreement.
- 3 Push fixes as new commits.**  
Do not amend. The reviewer needs to see what changed since they looked.
- 4 Disagree openly when you disagree.**  
“I kept it as is because X” is a fine outcome. A review is a conversation, not an order.
- 5 Re-request review when you are done.**  
Do not assume they are watching.

**THE WORST HABIT A STUDENT TEAM CAN DEVELOP**

Approving a teammate's PR in thirty seconds because they are your friend and you are busy.

This turns a real safety mechanism into a formality, and the first time it matters, nobody catches the bug. **If you do not have twenty minutes, say so and let someone else review.**

**TEAM RULE**

Every PR gets a real review from someone who did not write it, within **24 hours on a weekday**. The reviewer writes at least one substantive comment — a question counts.

"LGTM" alone is not a review, and the course deliverable explicitly requires *one review comment given to a teammate*.

**LIKELY VIVA QUESTION**

**"What do you look for in a code review?"**

Weak: "naming and formatting." Strong: the four checks — correctness, blast radius, test coverage, cognitive complexity — plus an example of a time you asked a question instead of assuming.

## 08

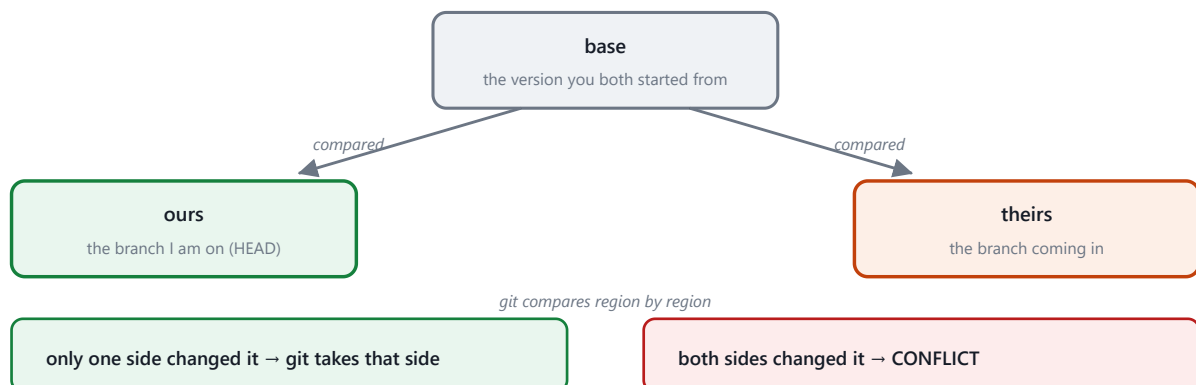
## Merge conflicts

A conflict is not git failing. It is git refusing to guess.

**In this part:** why conflicts happen · reading the markers · a full worked example · our 9-step procedure · how to have fewer conflicts

## 8.1 Why conflicts happen

Every merge starts with **three** versions of a file, not two.



Two edits, one region, no tie-breaker. That is the whole story.

Git compares *ours* against *base*, and *theirs* against *base*, one region at a time.

- If only one side touched a region, git takes that side automatically. **This is why most merges have no conflict at all.**
- If both sides touched the same region, git now holds two valid edits and **has no rule for choosing**. So it stops and asks you.

## 8.2 Reading the markers

Git writes both versions into the file and steps back.

```

<<<<<< HEAD
    return f"Hello, {name}!"
=====
    return f"Hi there, {name}."
>>>>>> feature/greeting
  
```



| Marker        | What it means                                                                 |
|---------------|-------------------------------------------------------------------------------|
| <<<<<< HEAD   | From here to the divider is <b>your branch's version</b> .                    |
| =====         | Just a divider. <b>No code</b> . It marks where yours ends and theirs begins. |
| >>>>>> branch | From the divider to here is <b>the incoming branch's version</b> .            |

**NEVER SHIP THE MARKERS**

**What you see:** the app crashes with a syntax error, or the page literally shows <<<<<< .

**Cause:** the markers were left in the file and committed.

**Always run this after resolving:**

```
$ git grep -n "<<<<<<" # must print nothing
```

## 8.3 A real conflict, resolved

**Setup.** Aaradhya and Divyanshi both edit the order-total function. Aaradhya adds a discount. Divyanshi adds tax. Same lines.

### WHAT THEY BOTH STARTED FROM

```
def order_total(items):
    subtotal = sum(i.price * i.qty for i in items)
    return subtotal
```

#### ours — main (Aaradhya merged first)

```
def order_total(items, discount=0):
    subtotal = sum(i.price * i.qty for i in
items)
    return subtotal - discount
```

#### theirs — feature/tax (Divyanshi)

```
def order_total(items, tax_rate=0.18):
    subtotal = sum(i.price * i.qty for i in
items)
    return subtotal * (1 + tax_rate)
```

### WHAT DIVYANSHI SEES

```
$ git switch feature/tax
$ git merge main
CONFLICT (content): Merge conflict in src/orders.py
Automatic merge failed; fix conflicts and then commit the result.
```

### THE THREE WRONG ANSWERS

| What people click        | Why it is wrong                                                                                             |
|--------------------------|-------------------------------------------------------------------------------------------------------------|
| "Accept current change"  | The discount silently disappears. Aaradhya's feature is gone and nobody notices until a customer complains. |
| "Accept incoming change" | The tax silently disappears. Same problem, other direction.                                                 |
| "Accept both changes"    | Two <code>return</code> statements. The second never runs. Often it still compiles.                         |

### THE RIGHT ANSWER — DECIDE THE BEHAVIOUR FIRST

*Is tax applied before or after the discount?* That is a **product question, not a git question.**

Divyanshi asks Aaradhya. They agree: discount first, then tax on the reduced amount.

```
def order_total(items, discount=0, tax_rate=0.18):
    subtotal = sum(i.price * i.qty for i in items)
    discounted = subtotal - discount
    return discounted * (1 + tax_rate)
```

```
$ git grep -n "<<<<<<"          # prints nothing ✓
$ pytest tests/test_orders.py    # both features still covered?
6 passed
$ git add src/orders.py
$ git commit                    # finishes the merge
```

**NOTICE WHAT HAPPENED**

Neither side's code survived unchanged. The answer is a **third version** that satisfies both intentions. That is what "resolve" means. It is almost never a choice between two buttons.

## 8.4 Our 9-step procedure

**TEAM RULE**

**Never resolve a conflict mechanically without understanding what the code is meant to do.**

If the other side's intent is not obvious from the code, stop and ask the person who wrote it. Clicking a button in the editor is not resolving — it is a coin flip with a compile step.

- 1 Stop.**  
Do not click anything yet. A conflict is information, not an emergency.
- 2 Understand both changes.**  
`git log --merge -p <file>` shows the commits from both sides. Read the messages — this is exactly why we write good ones.
- 3 Decide the intended behaviour.**  
What should the code do now that both features exist? Ask the other author if it is not obvious. Ten minutes of talking beats an hour of guessing.
- 4 Resolve by hand.**  
Write the version that satisfies both intentions. Often that is new code neither side wrote.
- 5 Remove every marker.**  
`git grep -n "<<<<<<"` must print nothing.
- 6 Run the tests.**  
Both sides' tests. If only one side has tests, add the missing one.
- 7 Review the result.**  
`git diff --cached`. Look at it as if it were someone else's PR.
- 8 Finish the merge.**  
`git add` the resolved files, then `git commit`.
- 9 Tell the other author.**  
In the PR: "I resolved a conflict in `orders.py`; we agreed discount applies before tax."

**IF YOU GET LOST MID-MERGE**

```
$ git merge --abort # puts everything back exactly as it was
```

Completely safe before you commit the merge. There is no shame in aborting, thinking, and starting again calmly.

## 8.5 How to have fewer conflicts

### Keep branches short

The longer a branch lives, the more of other people's work it must absorb at the end.

### Merge main into your branch daily

Ten small conflicts are far easier than one enormous one.

### Take turns on the same file

The lead decides who goes first (Part 6.5).

### Talk

"I am about to change `orders.py`" in the team chat prevents more conflicts than any tool.

### LIKELY VIVA QUESTION

#### "What causes a merge conflict?"

Weak answer: "two people edited the same file" — which is wrong, people do that all day without conflicts.

Strong answer: *"both sides changed the same region relative to the common ancestor, so the three-way merge has two valid edits and no rule for choosing."* The words **common ancestor** and **region** are what the examiner is listening for.

## 09

# .gitignore, config and secrets

The one mistake in this course that fixing the code will not undo.

In this part: config is an input, not a constant · what .gitignore is *not* · what to do when a secret is committed · the NEVER COMMIT list · the same rule for AI prompts

Hardcoded into source means hardcoded into git history — forever.

Lecture T0 · L0

## I 9.1 Config is an input, not a constant

This is the idea that makes everything else in this part work.

### X Config baked into the code

```
// config.js
const password = "Summer2024!";
```

This line *contains* a password. The moment it is committed it is in history, in every fork, and in every clone. Changing the database means editing source and redeploying — separately for every environment.

### ✓ Config read at startup

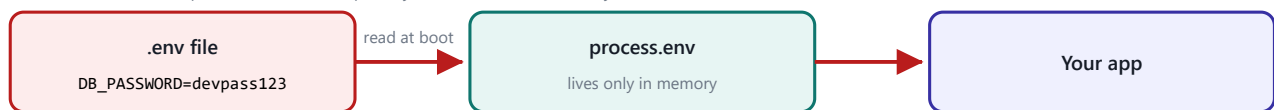
```
// config.js
const password = process.env.DB_PASSWORD;
```

This line never contains a password. It is an *instruction to go ask the environment* — and the environment answers differently in dev, staging and production. **The code never changes.**

**The Twelve-Factor App, Factor III — store config in the environment.** One codebase should deploy unchanged into any number of environments. Anything that differs between them — passwords, hostnames, feature flags — is an environment variable, never a checked-in file.

If changing behaviour means editing code, it is not configuration. It is a second copy of the program.

One file, read once, at startup. It never enters the repository and it never leaves memory.



## 9.2 What .gitignore is — and what it is not

`.gitignore` lists paths that should never reach the staging area. It **does not delete anything**. It just tells `git add` to skip those paths.

| Ignore this                                                                  | Why                                                                 |
|------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <code>node_modules/</code> , <code>.venv/</code> , <code>__pycache__/</code> | Huge, and can be regenerated. The lockfile records what to install. |
| <code>dist/</code> , <code>build/</code> , <code>*.pyc</code>                | Different on every machine. The build makes them again.             |
| <code>*.log</code> , <code>logs/</code>                                      | Machine-specific noise, and they can contain user data.             |
| <code>.env</code> , <code>*.pem</code> , <code>*.key</code>                  | Secrets. Never committed, never pushed.                             |
| <code>.vscode/</code> , <code>.idea/</code> , <code>.DS_Store</code>         | Personal preference, not team policy.                               |

```

# .gitignore

# dependencies
node_modules/
.venv/
__pycache__/

# build output
dist/
build/
*.pyc

# local logs
*.log
logs/

# secrets – never commit these
.env
.env.local
*.pem
*.key
secrets/

# editor / OS
.vscode/
.idea/
.DS_Store
  
```

**.GITIGNORE DOES NOT REMOVE SECRETS**

Adding `.env` to `.gitignore` **today** does nothing about a `.env` you committed **last week**. Git still has it, in history, forever.

Ignoring only stops *future* accidental staging.

**Prove it — do not just trust it**

```
$ cat .gitignore
.env
node_modules/

$ git log --all --full-history -- .env
✓ (no output – it was never committed, in any branch, ever)
```

Run this before every milestone submission. It takes one second and it is the difference between a clean hygiene score and a disaster.

## I 9.3 When a secret is already committed

The course tells this as a story. A small team hardcodes their production database password into a config file — faster, and it is Tuesday, nobody is looking. The file gets committed. The repo is public. **Within minutes, scanners built for exactly this find it.**

```
$ trufflehog filesystem . --only-verified
scanning 214 files across 1 repository...
X Found verified secret
  Reason: high-entropy string, matches known DB URI format
  File:   config.js:2
  Secret: DB_PASSWORD=Summer2024!
⚠ 1 verified secret found — rotate before doing anything else
```

### ROTATING DOES NOT UNDO THE EXPOSURE

The secret is already in the repo's history, in every fork, and in every clone. **You cannot recall a copy that someone else already has.** Rotating is still the first thing you do — because it makes the leaked value useless.

### What to do, in order

- 1 **Rotate the credential. Immediately.**  
Not after lunch. Make a new one and revoke the old one at the provider. **This is the only step that actually stops the bleeding.**
- 2 **Tell the team, the same hour.**  
Flagging a mistake early is treated far more leniently than one discovered later.
- 3 **Tell course staff, the same day.**  
The course requires the incident to be discussed, exactly as industry treats it.
- 4 **Remove it from the code.**  
Read it from the environment instead.
- 5 **Check what it could reach.**  
Was it read-only or admin? Look at the provider's access logs for anything you cannot explain.
- 6 **Write it in the failure log.**  
What leaked, when, how it was found, what was rotated.
- 7 **Only then think about rewriting history.**  
Tools like `git filter-repo` can strip it, but every later commit ID changes, everyone must re-clone, and **copies already taken still have the secret.** It is cleanup, not a fix.

## I 9.4 NEVER COMMIT THESE

Print this. It is the fastest check before any `git add`.

- |                                                                 |                                                                                                       |
|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> API keys of any kind                   | <input type="checkbox"/> Private keys ( <code>.pem</code> , <code>.key</code> , <code>id_rsa</code> ) |
| <input type="checkbox"/> Passwords — including “temporary” ones | <input type="checkbox"/> Production database connection strings                                       |



- ☐ Session or signing secrets
- ☐ OAuth client secrets and refresh tokens
- ☐ Cloud credentials (AWS, GCP, Azure)
- ☐ Service-account JSON files
- ☐ Personal data of real people
- ☐ Customer data of any kind
- ☐ Internal hostnames and IPs
- ☐ Bearer tokens copied from a browser
- ☐ Webhook signing secrets
- ☐ .env , .env.local , .env.production
- ☐ Database dumps with real rows
- ☐ Screenshots or logs with tokens visible

TEAM RULE

A credential exposed in a commit or in a prompt must be **rotated**, and the incident discussed. No member hides a leak to avoid embarrassment.

The course treats early disclosure far more leniently than a violation found later — and a defaulting member costs the whole team **25% of the affected grade**.

9.5 The same rule applies to AI prompts

Never put credentials, production connection strings or personal data into a prompt. This is a professional norm, and the course examines it. It is treated as an **academic-integrity failure — not a mistake**.

Worth answering as a team: *"Your assistant can read your whole working directory. Name three things it should never read, and what happens if it does."*

| It should never read                       | What happens if it does                                                                                                                                              |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| .env / .env.production                     | Live credentials enter the model's context, leave your machine, and may come back out in a chat log, a commit or a PR description. Every value must then be rotated. |
| Private keys and certificates              | Same exposure — but these often cannot be rotated quickly, because other systems depend on them.                                                                     |
| Database dumps or logs with real user data | Personal data of real people leaves the team's control. That is a privacy failure, and no rotation fixes it.                                                         |

## The mechanical defence — and its gap

```
// .claude/settings.json
{
  "permissions": {
    "deny": [
      "Read(**/.env*)",
      "Read(**/secrets/**)",
      "Read(**/*.pem)"
    ]
  }
}
```

### THE GOTCHA YOU WILL BE ASKED ABOUT

A `Read` deny rule blocks the **Read tool**. It does **not** stop `Bash("cat .env")` — same file, different door.

Real protection blocks both doors, or runs Bash in a sandbox that cannot see the path at all.

## 9.6 Three buckets, three different fates

### Team policy

#### Committed to git

`AGENTS.md`,  
`.claude/settings.json`. Build  
 commands, conventions,  
 shared deny rules. Everyone's  
 agent reads the same rules,  
 reviewed in PRs like any other  
 code.

### Personal preference

#### Gitignored, local

`.claude/settings.local.json`,  
`~/.claude/settings.json`.  
 Your own auto-approve list,  
 your own editor habits. **Never**  
**forced on a teammate.**

### Secrets

#### Never a file at all

API keys, DB passwords,  
 tokens. Environment variables  
 read at startup. **If it sits in a**  
**file the agent can open,**  
**assume it will be read.**

### LIKELY VIVA QUESTION

**"You accidentally committed an API key. What do you do?"**

Weak: "delete the file and commit again" — this shows you do not understand history.

Strong: **rotate the key first**, then disclose, then clean up — and say plainly that rewriting history does not recall copies other people already hold.

## 10

## Working with AI

Your assistant can write the code. It has no idea whether the code is right.

In this part: why it invents functions · context engineering · AGENTS.md · Plan → Spec → Generate → Validate · context rot · the AI code review checklist

## 10.1 Two ways to work with an assistant

### ✗ Vibe coding

- Loose prompts, fast iteration, judged by feel
- Great for a demo nobody depends on
- Falls apart the moment a second engineer, a real user, or real money shows up

### ✓ Spec-driven

- You write down the intent first
- You check the output against it
- You validate before it merges
- Slower on line one. The only kind that survives line ten thousand.

“It compiled. I shipped it. If you don’t know it’s doing the right thing, assume it’s doing the wrong.”

## 10.2 Why it invents functions that do not exist

This is the most useful idea in the whole topic. Once you understand it, you stop being surprised and start being careful.

| Step       | What actually happens                                                                                                                                                                                                                      |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tokenising | The model never reads a word. <code>getUserPermissions</code> becomes <code>get · User · Perm · issions</code> — pieces from a fixed vocabulary. <b>A function it has never seen still breaks into pieces it has seen a million times.</b> |
| Embedding  | Each piece becomes a list of numbers. Pieces used in similar ways end up near each other. Geometry stands in for meaning.                                                                                                                  |
| Attention  | Every token looks at every other token in the context. That is how it can use a function signature 400 lines away — <b>but only if that signature is in the context window.</b>                                                            |
| Sampling   | It produces a probability for every possible next token, and picks one. Then the next. <b>Nothing in that process knows your codebase unless your codebase is in the context.</b>                                                          |

**THERE IS NO LOOKUP STEP. ONLY A GUESS.**

Given `def check(user):` the model might produce `return user.is_admin` at 61% and `return check_role(user)` at 22%.

It is not looking in your repo and failing. **There is no looking step at all.** That is why a believable wrong answer is cheap to produce, and why “I don’t know” is not something it tends to say.

**The example from the lecture**

```
# routes/orders.py – generated, reads perfectly
def cancel_order(order_id):
    refund_service.issue_refund(order_id, reason="cancelled")
    return {"status": "cancelled"}
```

```
$ grep -rn "issue_refund" .
no matches found
$ grep -rn "class RefundService" .
no matches found
```

It looks right. `refund_service.issue_refund` reads exactly like a method that *should* exist. The whole module was invented — believably, once.

**RESTARTING THE CHAT DOES NOT FIX IT**

New chat, same repo, a brand new and **equally fake** `RefundService`.

The model is not remembering a mistake. It is making the same believable guess from the same missing information. The fix is not a fresh chat — it is **putting the real information into the context**.

**10.3 Context engineering****Prompt engineering**

Wording one message well. Phrasing, examples, tone.

**Scope:** the sentence you just typed.

**Context engineering**

System prompt, chat history, codebase rules, retrieved docs, tool output.

**Scope:** everything the model sees before it answers.

Four layers. Three of them are not yours.



**Most teams ship zero of these layers on purpose. The green one is the one you can fix today.**

## 10.4 The agent context file

| File      | Who it is for                                                                   | How often is it read?                      |
|-----------|---------------------------------------------------------------------------------|--------------------------------------------|
| README.md | Sells the project to a human.<br>Screenshots, badges, why you would want it.    | A person reads it once, then never again.  |
| AGENTS.md | Ground truth for the assistant. Test command, directory map, what not to touch. | Read at the start of every single session. |

CLAUDE.md is Anthropic’s own convention — read automatically, but tied to one assistant.

AGENTS.md is the open standard that Cursor, Windsurf, Copilot and Claude Code all read.

One file, every tool a teammate might open.

### TEAM RULE

We write AGENTS.md as the single source of truth, because the four of us may not use the same assistant. If a tool needs CLAUDE.md, we make it a one-line pointer to AGENTS.md rather than keeping two files that drift apart.

### Exact commands, not vague verbs

```
# AGENTS.md

## Build & test
pnpm test --watch=false
pnpm build

## Run locally
docker compose up          # app on :8000, postgres on :5432

## Directory map
src/api    - HTTP routes
src/db     - migrations and queries
src/ui     - components
tests/     - pytest; mirrors src/ layout

## Conventions
- Commits: type(scope): imperative subject
- Branches: feature/ fix/ refactor/ docs/ chore/
- All config read from process.env; add new vars to .env.example

## Rules
Never run db:push against schema.ts.
Edits to schema.ts must generate a migration.
Never read .env, secrets/, or any *.pem file.
```

Not “run the tests”. `pnpm test --watch=false` — the exact string it can paste into a shell.

**POSITIVE SUGGESTIONS GET SKIMMED. NEGATIVE RULES GET OBEYED.**

- ✗ "Please be careful with the database schema" — vague, skimmed.
- ✗ "Try to follow good database practices" — meaningless, skimmed.
- ✓ "Never run db:push. Edits to schema.ts must generate a migration." — *Never* and *must*. Obeyed.

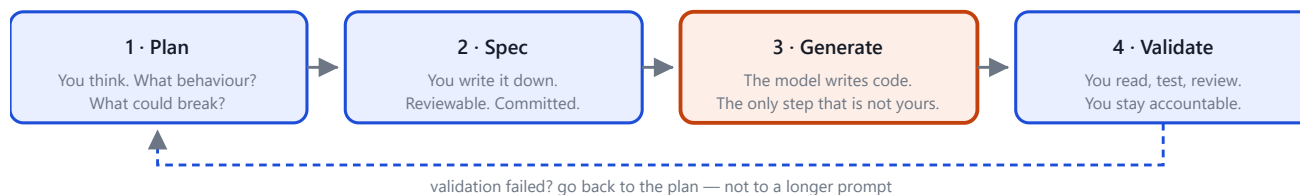
**A spec outlives the chat that made it**

| A specification is...      | ...whereas a prompt is      |
|----------------------------|-----------------------------|
| Version-controlled         | Gone when the chat closes   |
| Reviewable by a teammate   | Stuck in one person's head  |
| Permanent                  | Tied to one conversation    |
| Comparable across attempts | A one-off you cannot re-run |

The job changed. Before: your main job was generating code. Now: your job is describing intent precisely, and checking what came back.

**10.5 Plan → Spec → Generate → Validate**

Only step 3 is where the model writes code. Steps 1, 2 and 4 are yours — and they decide whether step 3 is worth trusting.

**How we run it on a real task**

- 1 Understand the requirement**  
In your own words, before opening any chat. If you cannot state the behaviour, no prompt will save you.
- 2 Think about the design**  
Where does this live? What does it touch? What are the edge cases? Three lines in the issue.
- 3 Ask, with the spec and the real files**  
Not "write me a cancel feature".
- 4 Read the whole diff**  
Every import, every function call. Then `grep` for anything you do not recognise.
- 5 Understand every important line**  
If you cannot explain a line to a teammate, it does not go in.

**6 Test it**

Run the suite, and write a test if the model did not.

**7 Security check**

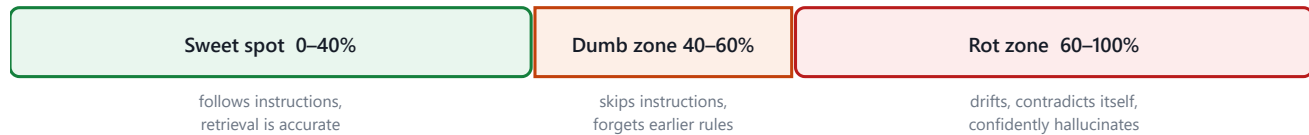
The checklist in 10.7.

**8 Commit, open the PR, get a human review**

Say in the description which parts were AI-assisted and what you verified.

## 10.6 Context rot

What happens to a chat after 50 turns? **It gets worse, not better.**



Dex Horthy (HumanLayer) named the middle band the “dumb zone”.

### THE GOTCHA

These zones are **absolute token counts, not percentages**. A 1M-token window at 40% has already burned 400K tokens — deep into the range where quality drops.

**A bigger window buys you more room. It does not move the wall.**

And every kept token costs. Attention compares every token to every other one, so cost grows with the **square** of the context. Double the context, quadruple the comparisons. **A 50-turn chat is not just noisier — it is measurably slower and more expensive per reply.**

**The fix is not a bigger context window. It is a shorter one.**

### WHAT WE DO ABOUT IT

- **One task, one session.** Finish the feature, then start fresh for the next thing.
- **Put lasting facts in `AGENTS.md`**, not in the chat. The file is re-read every session; the chat is not.
- **Use subagents to explore.** A subagent reads the whole repo in its own context and returns *only its conclusion*. The noise is thrown away.
- **Restart when answers start contradicting earlier ones.** That is the rot zone. Re-prompting will not fix it.

## 10.7 What to use it for — and what to check

### ✓ Good uses

- Explaining unfamiliar code
- Boilerplate and scaffolding
- First-draft tests for edge cases you list
- Debugging: “what could cause this stack trace?”
- Mechanical refactoring
- Documentation drafts
- A second pair of eyes on your own diff

### ✗ Never trust blindly

- Whether a function exists in *your* repo
- Security decisions — auth, crypto, input checks
- Whether the tests assert anything useful
- The real root cause, without you confirming it
- Anything touching a database schema
- Library versions and API signatures
- Replacing your teammate’s review



## Weak prompt vs strong prompt

### X Weak

add order cancellation

No behaviour, no constraints, no context. You will get a believable module that calls things which do not exist.

### ✓ Strong

In `src/api/orders.py`, add POST `/orders/:id/cancel`.

#### Behaviour:

- only the order's owner may cancel
- reject if status is 'shipped' (409)
- calling twice does nothing, returns 200
- sets `cancellation_requested_at`

#### Constraints:

- use the existing `OrderRepo` in `src/db/orders.py`
- parameterised queries only
- no new dependencies

Then write pytest cases for: happy path, wrong user, already shipped, double cancel.

**The pattern:** where the code goes, what the behaviour is, the edge cases, what it must use, what it must not do, and what tests you expect. That is a *spec* — which is why it is reviewable and repeatable.

## 10.8 AI code review checklist

Run this on every piece of AI-generated code **before** you commit it. Five minutes. It is the difference between using a tool and being used by one.

### DOES IT EXIST?

- ☐ Every imported module is really installed and in the lockfile
- ☐ Every function and class it calls actually exists — I grepped for the ones I did not recognise
- ☐ Every API signature matches the real docs, not the model's memory of them
- ☐ No new dependency was added without the team agreeing

### DOES IT DO THE RIGHT THING?

- ☐ I can explain every important line out loud, without the chat open
- ☐ It does what I specified — not a nearby, easier thing
- ☐ Edge cases are handled: empty, zero, one, maximum, duplicate, concurrent
- ☐ Failures do something sensible, not just `except: pass`
- ☐ It matches our conventions instead of importing a foreign style

### IS IT SAFE?

- ☐ All queries are parameterised — no SQL built from strings
- ☐ All user input is checked before use

- ☐ Authorisation is checked for *this specific object*, not just “is logged in”
- ☐ No secret, key or connection string is hardcoded
- ☐ Errors do not leak internal details to the user
- ☐ Nothing logs a password, token or personal data

IS IT OURS?

- ☐ Tests exist and I ran them
- ☐ The commit is in my name and I stand behind it
- ☐ The PR says which parts were AI-assisted and what I verified
- ☐ No production credential or personal data ever went into a prompt

TEAM RULE

**You remain accountable for everything you submit.** “The assistant produced it” is not a defence — in this course or in a job.

Practically: **if you cannot explain it in the PR, you do not open the PR.**

10.9 Learning, not depending

AI is allowed on practical work and milestones. It is **not** allowed in quizzes, individual exercises or the capstone defence. The gap between those two is exactly the gap between using AI to learn and using it to avoid learning.

| Builds dependency                       | Builds understanding                                                                     |
|-----------------------------------------|------------------------------------------------------------------------------------------|
| “Write this feature.” Paste. Move on.   | “Write this feature.” Then “why this approach over X?” Then read it and decide yourself. |
| Asking for the fix to an error          | Asking what the error <i>means</i> , predicting the fix, then checking                   |
| Accepting the first answer              | Asking for two approaches and their trade-offs, then choosing and writing down why       |
| Letting it write the tests and the code | Writing the test list yourself; letting it write the implementation                      |
| Never reading the diff                  | Reading the diff before running it, every time                                           |

WHY THIS MATTERS AT THE END

The capstone puts you in an **unfamiliar repository with no assistant**: bring it up, find a bug, fix it, open a PR.

Everything in this part is training for that moment. A student who has read every diff all semester walks in calm. A student who has pasted all semester does not.